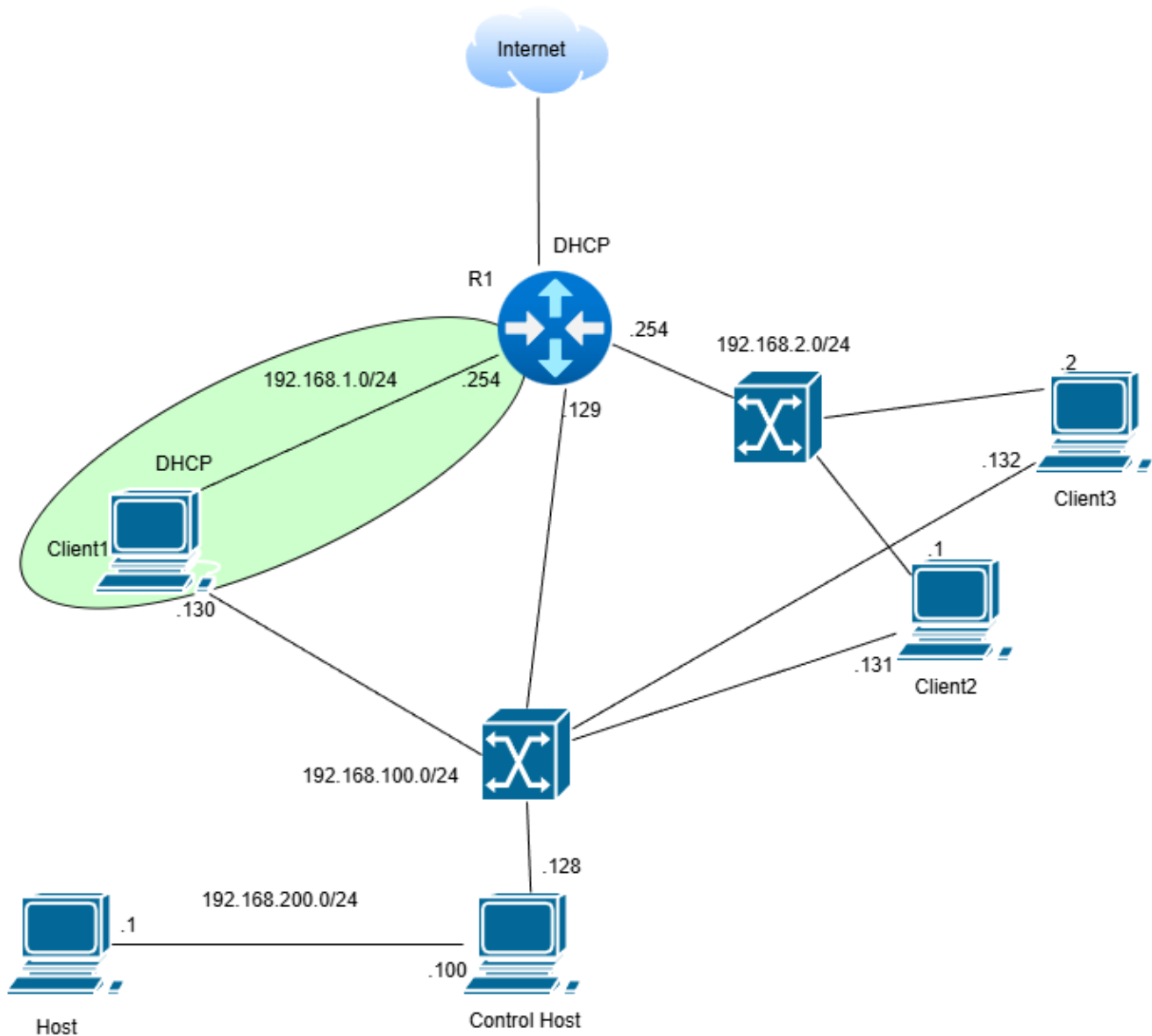


WireGuard Dual-Stack VPN

TOPO



Plan

Aufbauend auf einer bestehenden Netzwerkumgebung (z. B. aus vorherigen BS-Aufgaben).

Wir konfigurieren einen WireGuard Dual-Stack (IPv4 + IPv6) VPN-Tunnel zwischen Client1 und Client3. Die Konfiguration wird über ein Ansible-Skript automatisiert, das WireGuard installiert, Schlüssel generiert und die Konfigurationsdateien erstellt. Client3 agiert als Listener (Server-Seite), Client1 als Connector (Client-Seite).

- VPN-Subnetz: IPv4 10.0.0.0/24, IPv6 fd0d:86fa:c3bc::/64
- Endpoint: Client3 hört auf Port 51820 (IP: 192.168.2.2)

Nach der Konfiguration testen wir den Durchsatz mit iperf und die CPU-Auslastung mit htop.

Client3 (Listener)

Interface	IP-Adressen	Rolle
wg0	10.0.0.1/24, fd0d:86fa:c3bc::1/64	VPN-Interface
ens38	192.168.2.2 (Beispiel-IP)	Extern

Konfiguration (via Ansible-Skript)

- Installiere WireGuard: `apt install wireguard`
- Erstelle Verzeichnis: `/etc/wireguard` mit Modus 0700
- Generiere privaten Schlüssel: `wg genkey > /etc/wireguard/private.key` (mit umask 077)
- Setze Berechtigungen: `chmod 0600 /etc/wireguard/private.key`
- Generiere öffentlichen Schlüssel: `cat /etc/wireguard/private.key | wg pubkey > /etc/wireguard/public.key`
- Hole öffentlichen Schlüssel von Client1 (via Ansible slurp und delegate_to)
- Erstelle Konfigurationsdatei `/etc/wireguard/wg0.conf`: `[Interface] Address = 10.0.0.2/24, fd0d:86fa:c3bc::2/64 PrivateKey = <Inhalt von private.key> [Peer] PublicKey = <Öffentlicher Schlüssel von Client3> AllowedIPs = 10.0.0.1/32, fd0d:86fa:c3bc::1/128 Endpoint = 192.168.2.2:51820 PersistentKeepalive = 25`
- Setze Berechtigungen: `chmod 0600 /etc/wireguard/wg0.conf`
- Starte Service: `systemctl enable --now wg-quick@wg0`

Nach Konfiguration

- Überprüfe Status: `wg show`
- Jetzt **TESTEN**

Client1 (Connector)

Interface	IP-Adressen	Rolle
wg0	10.0.0.2/24, fd0d:86fa:c3bc::2/64	VPN-Interface
ens37	192.168.1.x (Beispiel-IP)	Extern

Konfiguration (via Ansible-Skript)

- Installiere WireGuard: `apt install wireguard`
- Erstelle Verzeichnis: `/etc/wireguard` mit Modus 0700
- Generiere privaten Schlüssel: `wg genkey > /etc/wireguard/private.key` (mit umask 077)
- Setze Berechtigungen: `chmod 0600 /etc/wireguard/private.key`
- Generiere öffentlichen Schlüssel: `cat /etc/wireguard/private.key | wg pubkey > /etc/wireguard/public.key`
- Hole öffentlichen Schlüssel von Client3 (via Ansible slurp und delegate_to)
- Erstelle Konfigurationsdatei `/etc/wireguard/wg0.conf`: `[Interface] Address = 10.0.0.1/24, fd0d:86fa:c3bc::1/64 ListenPort = 51820 PrivateKey = <Inhalt von private.key> [Peer] PublicKey = <Öffentlicher Schlüssel von Client1> AllowedIPs = 10.0.0.2/32, fd0d:86fa:c3bc::2/128`
- Setze Berechtigungen: `chmod 0600 /etc/wireguard/wg0.conf`

- Starte Service: `systemctl enable --now wg-quick@wg0`

Nach Konfiguration

- Überprüfe Status: `wg show`
- Stelle sicher, dass der Tunnel aufbaut (Ping über VPN-IPs testen)
- Jetzt [TESTEN](#)

Router (Ubuntu mit GUI)

Falls ein Router dazwischen benötigt wird (z. B. aus BS1.1), stelle sicher, dass Routing für die VPN-Subnetze aktiviert ist und Firewalls UDP-Port 51820 durchlassen.

Interface	IP	LAN Segment
ens37	192.168.1.254	Lan_1-1
ens38	192.168.2.254	Lan_2-1

Konfiguration

- IP-Konfiguration wie in der Tabelle (via GUI oder netplan)
- Routing aktivieren: Entkommentiere `net.ipv4.ip_forward=1` in `/etc/sysctl.conf` und lade mit `sysctl -p`
- Teste Ping zwischen Client1 und Client3 über externe IPs
- Eventuell Firewall anpassen: Erlaube UDP 51820

Testen

Tunnel-Verbindung Testen

- Auf beiden Clients: `wg show` (sollte Peers und Handshakes anzeigen)

```
administrator@Client1:~$ sudo wg show
sudo: unable to resolve host Client1: Name or service not known
[sudo] password for administrator:
interface: wg0
  public key: pFt/lfRdmc3205dThgL57KzKGoCqb7Xh5uxYB3zGWVI=
  private key: (hidden)
  listening port: 34137

peer: zuk/5U/oeYwRA6ASFKw92ckgGmanLQYBM7pGik9QjCA=
  endpoint: 192.168.2.2:51820
  allowed ips: 10.0.0.1/32, fd0d:86fa:c3bc::1/128
  latest handshake: 36 seconds ago
  transfer: 28.25 MiB received, 2.83 GiB sent
  persistent keepalive: every 25 seconds
administrator@Client1:~$ _
```

```

administrator@Client3:~$ sudo wg show
[sudo] password for administrator:
interface: wg0
  public key: zuk/5U/oeYwrA6ASFKw92ckgGmanLQYBM7pGik9QjCA=
  private key: (hidden)
  listening port: 51820

peer: pFt/1fRdmc3205dThgL57KzKGoCqb7Xh5uxYB3zGHWI=
  endpoint: 192.168.1.150:34137
  allowed ips: 10.0.0.2/32, fd0d:86fa:c3bc::2/128
  latest handshake: 44 seconds ago
  transfer: 212 B received, 92 B sent
administrator@Client3:~$

```

Bild zeigt den Status des WireGuard-Interfaces und verbundene Peers.

- Ping-Test: Von Client1 zu Client3 über VPN-IPs (z. B. `ping 10.0.0.1` und `ping fd0d:86fa:c3bc::1`)

```

administrator@Client1:~$ ping 10.0.0.1 -t 5
PING 10.0.0.1 (10.0.0.1) 56(84) bytes of data.
64 bytes from 10.0.0.1: icmp_seq=1 ttl=64 time=0.893 ms
64 bytes from 10.0.0.1: icmp_seq=2 ttl=64 time=1.02 ms
64 bytes from 10.0.0.1: icmp_seq=3 ttl=64 time=1.51 ms
64 bytes from 10.0.0.1: icmp_seq=4 ttl=64 time=0.924 ms
64 bytes from 10.0.0.1: icmp_seq=5 ttl=64 time=0.842 ms
64 bytes from 10.0.0.1: icmp_seq=6 ttl=64 time=0.848 ms
64 bytes from 10.0.0.1: icmp_seq=7 ttl=64 time=0.851 ms
^C
--- 10.0.0.1 ping statistics ---
7 packets transmitted, 7 received, 0% packet loss, time 6013ms
rtt min/avg/max/mdev = 0.842/0.983/1.510/0.222 ms
administrator@Client1:~$ ping fd0d:86fa:c3bc::1
PING fd0d:86fa:c3bc::1 (fd0d:86fa:c3bc::1) 56 data bytes
64 bytes from fd0d:86fa:c3bc::1: icmp_seq=1 ttl=64 time=1.10 ms
64 bytes from fd0d:86fa:c3bc::1: icmp_seq=2 ttl=64 time=0.911 ms
64 bytes from fd0d:86fa:c3bc::1: icmp_seq=3 ttl=64 time=0.816 ms
64 bytes from fd0d:86fa:c3bc::1: icmp_seq=4 ttl=64 time=0.903 ms
64 bytes from fd0d:86fa:c3bc::1: icmp_seq=5 ttl=64 time=1.17 ms
64 bytes from fd0d:86fa:c3bc::1: icmp_seq=6 ttl=64 time=0.780 ms
^C
--- fd0d:86fa:c3bc::1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5013ms
rtt min/avg/max/mdev = 0.780/0.945/1.165/0.140 ms
administrator@Client1:~$

```

```
administrator@Client3:~$ ping 10.0.0.2
PING 10.0.0.2 (10.0.0.2) 56(84) bytes of data.
64 bytes from 10.0.0.2: icmp_seq=1 ttl=64 time=0.984 ms
64 bytes from 10.0.0.2: icmp_seq=2 ttl=64 time=0.999 ms
64 bytes from 10.0.0.2: icmp_seq=3 ttl=64 time=1.06 ms
64 bytes from 10.0.0.2: icmp_seq=4 ttl=64 time=0.904 ms
64 bytes from 10.0.0.2: icmp_seq=5 ttl=64 time=0.885 ms
^C
--- 10.0.0.2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4009ms
rtt min/avg/max/mdev = 0.885/0.966/1.062/0.064 ms
administrator@Client3:~$ ping fd0d:86fa:c3bc::2
PING fd0d:86fa:c3bc::2 (fd0d:86fa:c3bc::2) 56 data bytes
64 bytes from fd0d:86fa:c3bc::2: icmp_seq=1 ttl=64 time=0.947 ms
64 bytes from fd0d:86fa:c3bc::2: icmp_seq=2 ttl=64 time=0.953 ms
64 bytes from fd0d:86fa:c3bc::2: icmp_seq=3 ttl=64 time=0.888 ms
64 bytes from fd0d:86fa:c3bc::2: icmp_seq=4 ttl=64 time=0.860 ms
64 bytes from fd0d:86fa:c3bc::2: icmp_seq=5 ttl=64 time=0.931 ms
^C
--- fd0d:86fa:c3bc::2 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4007ms
rtt min/avg/max/mdev = 0.860/0.915/0.953/0.035 ms
```

Durchsatz Testen mit iperf

- Installiere iperf auf beiden: `apt install iperf3`
- Auf Client3 (Server): `iperf3 -s` (IPv4)
- Auf Client1 (Client): `iperf3 -c 10.0.0.1 -P 4`

- Erwarteter Output: Zeigt Bandbreite in Mbits/sec (z. B. 100-500 Mbits/sec je nach Netzwerk)

```

administrator@Client1:~$ iperf3 -c 10.0.0.1 -P 4
Connecting to host 10.0.0.1, port 5201
[ 5] local 10.0.0.2 port 57130 connected to 10.0.0.1 port 5201
[ 7] local 10.0.0.2 port 57132 connected to 10.0.0.1 port 5201
[ 9] local 10.0.0.2 port 57136 connected to 10.0.0.1 port 5201
[11] local 10.0.0.2 port 57140 connected to 10.0.0.1 port 5201
[ ID] Interval           Transfer     Bitrate      Retr   Cwnd
[ 5]   0.00-1.00       sec    7.12 MBytes  59.7 Mbits/sec    51   54.8 KBytes
[ 7]   0.00-1.00       sec   14.1 MBytes  118 Mbits/sec    51   106 KBytes
[ 9]   0.00-1.00       sec    7.00 MBytes  58.7 Mbits/sec    57   58.8 KBytes
[11]   0.00-1.00       sec   11.6 MBytes  97.4 Mbits/sec    99   65.5 KBytes
[SUM] 0.00-1.00       sec   39.9 MBytes  334 Mbits/sec   258
- - - - -
[ 5]   1.00-2.00       sec    8.25 MBytes  69.2 Mbits/sec    74   58.8 KBytes
[ 7]   1.00-2.00       sec   11.4 MBytes  95.4 Mbits/sec    74   64.1 KBytes
[ 9]   1.00-2.00       sec   12.0 MBytes  101 Mbits/sec    40   89.5 KBytes
[11]   1.00-2.00       sec    8.75 MBytes  73.4 Mbits/sec   109   25.4 KBytes
[SUM] 1.00-2.00       sec   40.4 MBytes  339 Mbits/sec   297
- - - - -
[ 5]   2.00-3.00       sec    9.50 MBytes  79.7 Mbits/sec    45   41.4 KBytes
[ 7]   2.00-3.00       sec   13.8 MBytes  115 Mbits/sec    33   58.8 KBytes
[ 9]   2.00-3.00       sec    9.38 MBytes  78.6 Mbits/sec    49   62.8 KBytes
[11]   2.00-3.00       sec    9.38 MBytes  78.6 Mbits/sec    18   62.8 KBytes
[SUM] 2.00-3.00       sec   42.0 MBytes  352 Mbits/sec   145
^C- - - - -
[ 5]   3.00-3.28       sec    2.75 MBytes  81.7 Mbits/sec     6   50.8 KBytes
[ 7]   3.00-3.28       sec    2.75 MBytes  81.7 Mbits/sec     5   54.8 KBytes
[ 9]   3.00-3.28       sec    3.25 MBytes  96.5 Mbits/sec    33   46.8 KBytes
[11]   3.00-3.28       sec    2.62 MBytes  77.9 Mbits/sec    17   52.1 KBytes
[SUM] 3.00-3.28       sec   11.4 MBytes  338 Mbits/sec    61
- - - - -
[ ID] Interval           Transfer     Bitrate      Retr
[ 5]   0.00-3.28       sec   27.8 MBytes  70.9 Mbits/sec   176      sender
[ 5]   0.00-3.28       sec    0.00 Bytes  0.00 bits/sec              receiver
[ 7]   0.00-3.28       sec   42.0 MBytes  107 Mbits/sec   163      sender
[ 7]   0.00-3.28       sec    0.00 Bytes  0.00 bits/sec              receiver
[ 9]   0.00-3.28       sec   31.8 MBytes  81.1 Mbits/sec   179      sender
[ 9]   0.00-3.28       sec    0.00 Bytes  0.00 bits/sec              receiver
[11]   0.00-3.28       sec   32.4 MBytes  82.7 Mbits/sec   243      sender
[11]   0.00-3.28       sec    0.00 Bytes  0.00 bits/sec              receiver
[SUM] 0.00-3.28       sec   134 MBytes  342 Mbits/sec   761      sender
[SUM] 0.00-3.28       sec    0.00 Bytes  0.00 bits/sec              receiver
iperf3: interrupt - the client has terminated

```

Bild zeigt den gemessenen Datendurchsatz über den VPN-Tunnel. Wie wir hier sehen können macht es keinen deutlichen Unterschied im throughput.

- Installiere htop: `sudo apt install htop`
- Während iperf läuft: Starte `htop` auf beiden Clients
- Überwache CPU-Nutzung des iperf-Prozesses (sollte niedrig sein, z. B. <10% auf moderner Hardware)

Main		I/O											
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command		
1664	iperf3	20	0	42516	4832	3712	R	6.6	0.1	0:00.16	/usr/bin/iperf3 --server --interval 0		
1665	iperf3	20	0	42516	4832	3712	R	6.6	0.1	0:00.17	/usr/bin/iperf3 --server --interval 0		
1666	iperf3	20	0	42516	4832	3712	S	5.8	0.1	0:00.15	/usr/bin/iperf3 --server --interval 0		
1667	iperf3	20	0	42516	4832	3712	S	5.8	0.1	0:00.16	/usr/bin/iperf3 --server --interval 0		

Bild zeigt die CPU-Auslastung während des iperf-Tests. Hier sehen wir zwar einen kleinen aber dennoch beträchtlichen unterschied. Während der iperf test durch die öffentliche/external IP eine auslastung von 6.6% zeigt, sehen wir, dass die Auslastung beim VPN-Tunnel 3.7% beträgt.